ML_TASK_6(SVM_with_PCA)

June 12, 2026

1 Apply SVM Algorithms by using different kernels and Evaluate the model

[1]:
```
# ======================================================
# IMPORT LIBRARIES
# ======================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import (
    train_test_split,
    cross_val_score,
    GridSearchCV
)

from sklearn.svm import SVC

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)
```

[2]:
```
# ======================================================
# LOAD DATASET
# ======================================================

cancer = load_breast_cancer()
```

```
df = pd.DataFrame(
    cancer.data,
    columns=cancer.feature_names
)

df["diagnosis"] = cancer.target
```

1.1 Target and Fearure Selection

[3]:
```
# ====================================================
# FEATURES & TARGET
# ====================================================

X = df.drop("diagnosis", axis=1)

y = df["diagnosis"]
```

1.2 Exploratory Data Analysis

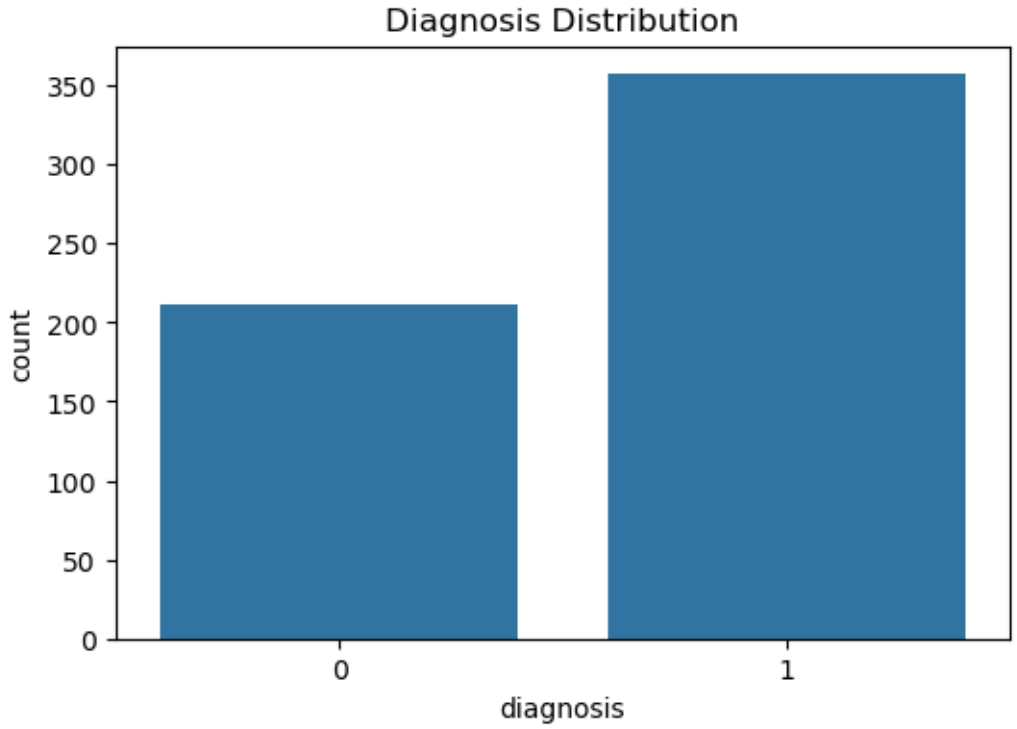
[4]:
```
print(cancer.data.shape)

print("Shape:", df.shape)

print("\nFirst 5 Rows")
print(df.head())

print("\nTarget Classes")
print(cancer.target_names)


print(cancer.target_names[0]) # 0 is negative and its malignant
print(cancer.target_names[1])  # 1 is positive and shows Benign
```

```
(569, 30)
Shape: (569, 31)

First 5 Rows
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0        17.99         10.38          122.80     1001.0          0.11840
1        20.57         17.77          132.90     1326.0          0.08474
2        19.69         21.25          130.00     1203.0          0.10960
3        11.42         20.38           77.58      386.1          0.14250
4        20.29         14.34          135.10     1297.0          0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry  \
0           0.27760          0.3001              0.14710         0.2419
```

```
1           0.07864          0.0869            0.07017         0.1812
2           0.15990          0.1974            0.12790         0.2069
3           0.28390          0.2414            0.10520         0.2597
4           0.13280          0.1980            0.10430         0.1809

   mean fractal dimension  …  worst texture  worst perimeter  worst area  \
0                 0.07871  …          17.33           184.60      2019.0
1                 0.05667  …          23.41           158.80      1956.0
2                 0.05999  …          25.53           152.50      1709.0
3                 0.09744  …          26.50            98.87       567.7
4                 0.05883  …          16.67           152.20      1575.0

   worst smoothness  worst compactness  worst concavity  worst concave points  \
0            0.1622             0.6656           0.7119                0.2654
1            0.1238             0.1866           0.2416                0.1860
2            0.1444             0.4245           0.4504                0.2430
3            0.2098             0.8663           0.6869                0.2575
4            0.1374             0.2050           0.4000                0.1625

   worst symmetry  worst fractal dimension  diagnosis
0          0.4601                  0.11890          0
1          0.2750                  0.08902          0
2          0.3613                  0.08758          0
3          0.6638                  0.17300          0
4          0.2364                  0.07678          0

[5 rows x 31 columns]

Target Classes
['malignant' 'benign']
malignant
benign
```

[5]:
```
# ---------------------------
# Diagnosis Count Plot
# ---------------------------

plt.figure(figsize=(6,4))

sns.countplot(
    x='diagnosis',
    data=df
)

plt.title("Diagnosis Distribution")

plt.savefig(
```

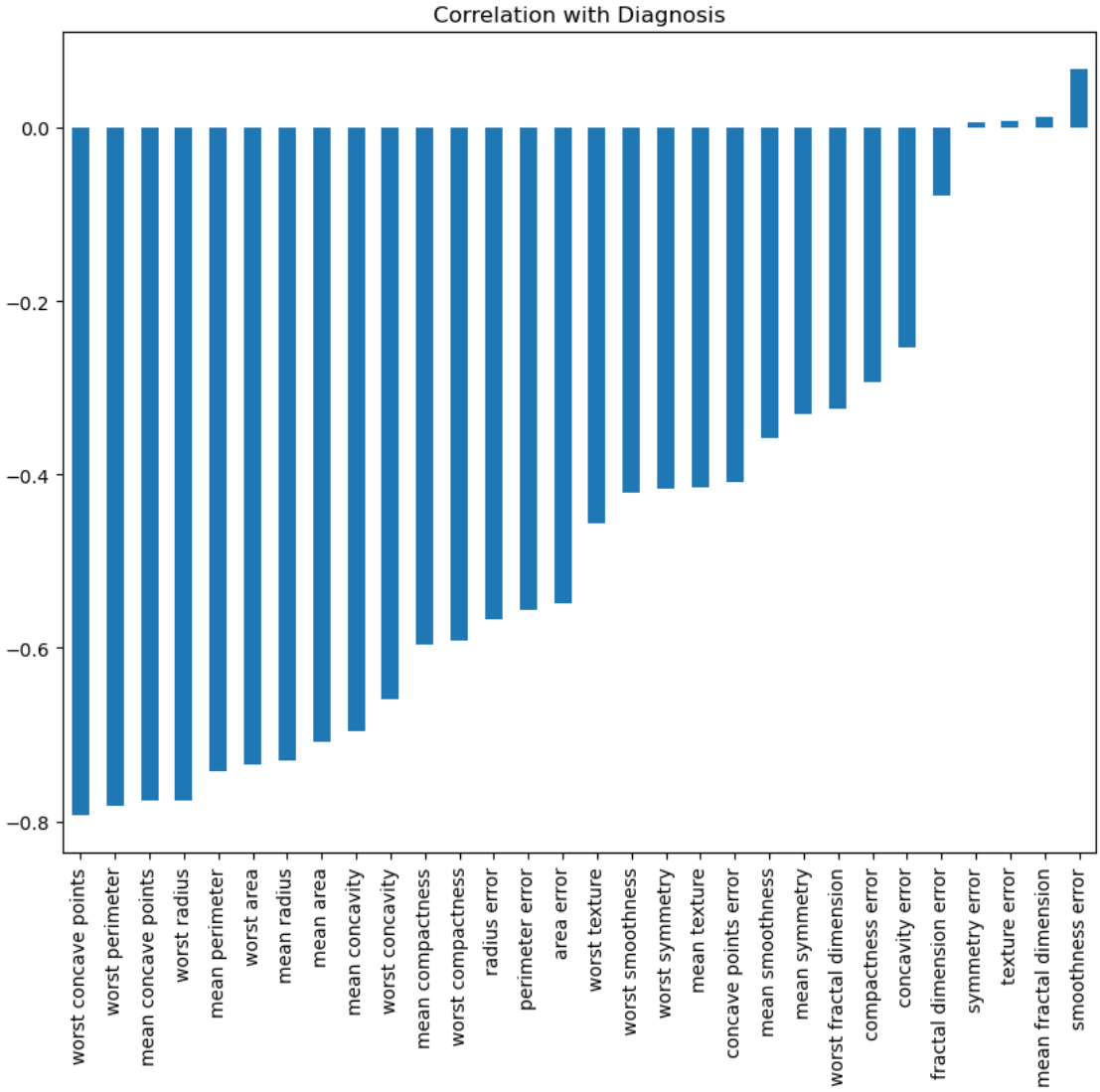
```
    "diagnosis_distribution.png",
    dpi=300,
    bbox_inches='tight'
)
plt.show()
```



The diagnosis distribution plot shows that the dataset contains 357 benign and 212 malignant samples. Although the classes are not perfectly balanced, the imbalance is moderate and suitable for classification analysis using stratified validation techniques.

[6]:
```
# ---------------------------
# Correlation Plot
# ---------------------------

plt.figure(figsize=(10,8))

df.corr()['diagnosis'][:-1].sort_values().plot(
    kind='bar'
)
plt.title("Correlation with Diagnosis")

plt.savefig(
    "correlation_with_diagnosis.png",
```

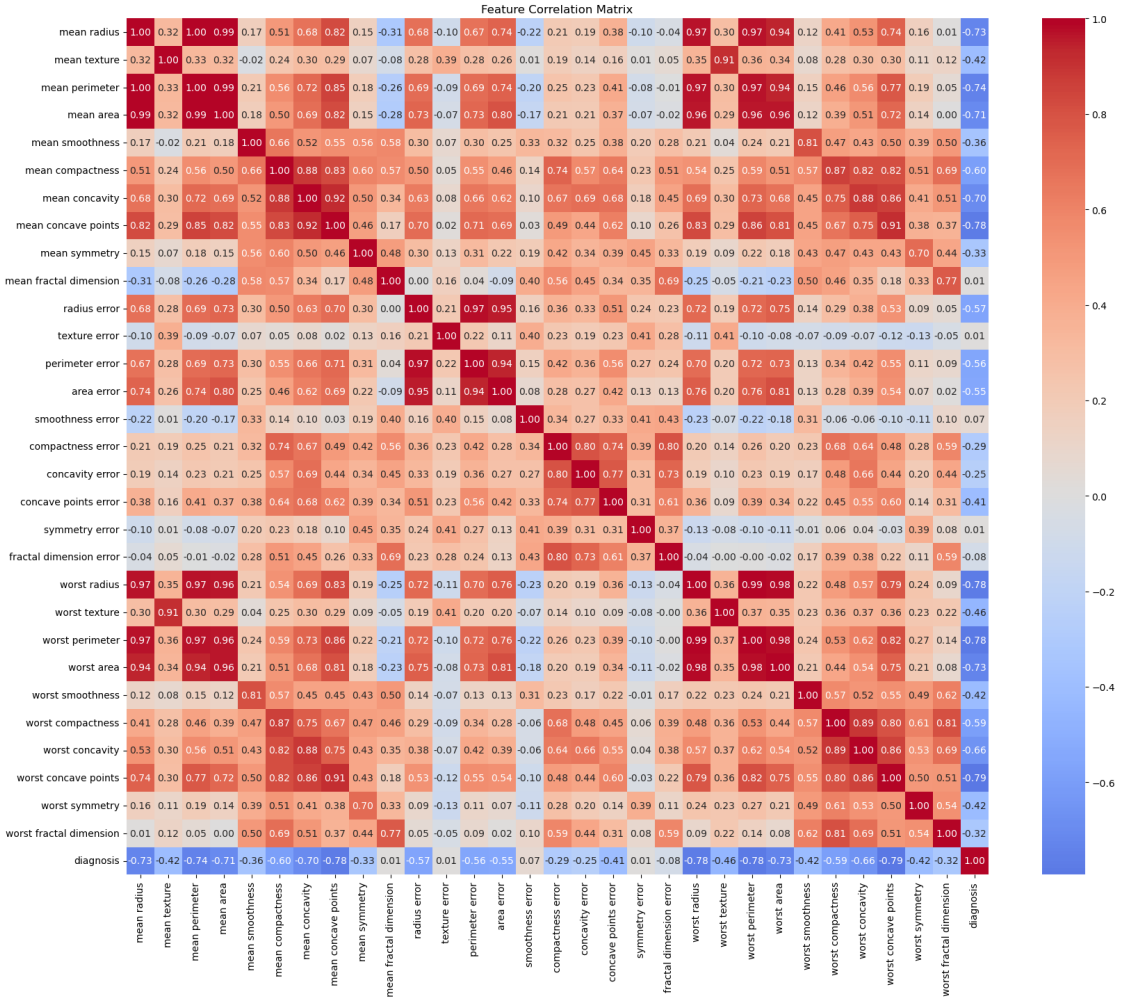
```
    dpi=500,
    bbox_inches='tight'
)
```



The correlation analysis reveals that features such as worst concave points, worst perimeter, mean concave points, and worst radius have the strongest association with the diagnosis. These features are likely to be the most important predictors for distinguishing between benign and malignant tumors.

[7]:
```
plt.figure(figsize=(20,16))

# Correlation matrix
corr_matrix = df.corr(numeric_only=True)
```

```
sns.heatmap(
    corr_matrix,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    center=0
)

plt.title("Feature Correlation Matrix")
plt.savefig(
    "feature_correlation_matrix.png",
    dpi=300,
    bbox_inches='tight'
)
plt.show()
```



The correlation heatmap shows strong positive correlations among several features, particularly radius, perimeter, area, and concavity-related measurements. This indicates the presence of multicollinearity and redundant information in the dataset, which justifies the use of PCA for dimensionality reduction.

2 Feature Scaling and PCA

[8]:
```
# ======================================================
# FEATURE SCALING
# ======================================================

#We do standard scaloing which result in mean 0 and variance 1
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# ======================================================
# Apply PCA as well
# ======================================================

pca = PCA(n_components=0.95)

X_pca = pca.fit_transform(X_scaled)

print("\nOriginal Features:", X.shape[1])

print("Reduced Features:", X_pca.shape[1])

print("\nExplained Variance Ratio")

print(pca.explained_variance_ratio_)


print("Explained Variance:",
      pca.explained_variance_ratio_.sum())
```

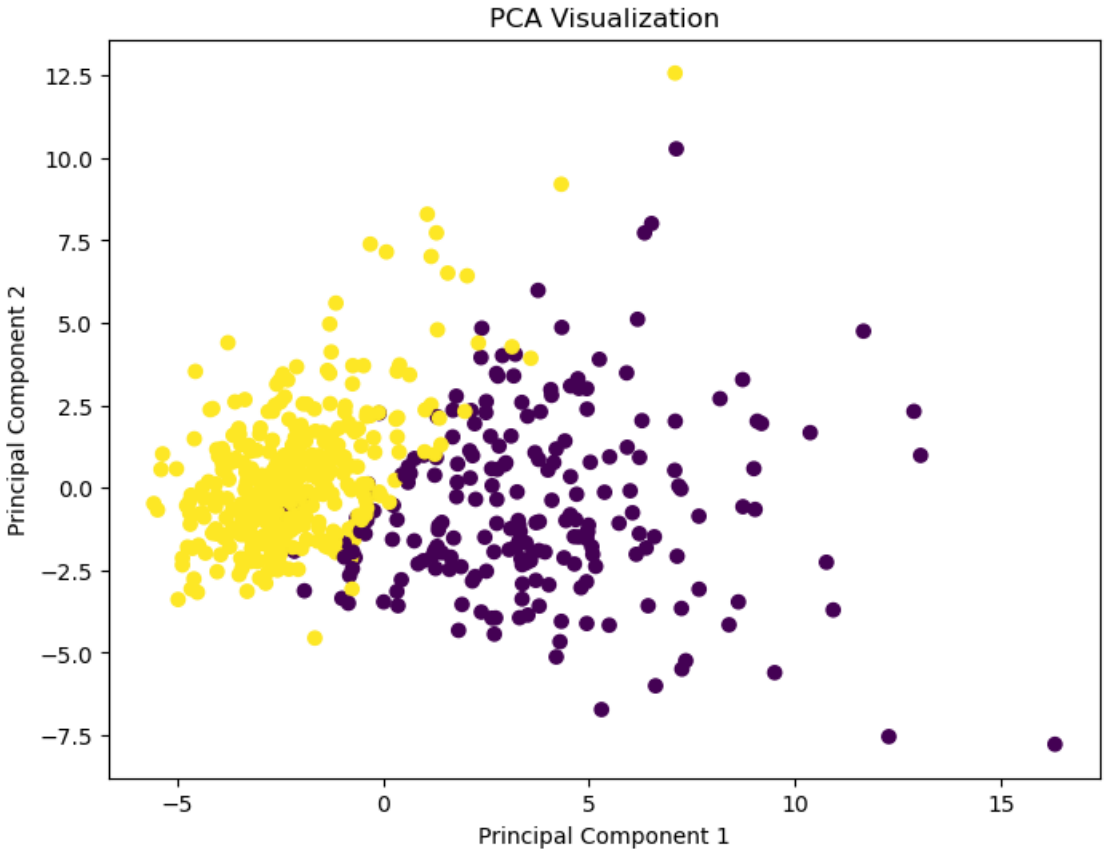
```
Original Features: 30
Reduced Features: 10

Explained Variance Ratio
[0.44272026 0.18971182 0.09393163 0.06602135 0.05495768 0.04024522
 0.02250734 0.01588724 0.01389649 0.01168978]
Explained Variance: 0.9515688143366668
```

PCA selected the minimum number of principal components needed to preserve at least 95% of the total variance. In this dataset, the first **10** principal components captured about **95.15%** of the information present in the original 30 features, so the remaining 20 components contributed very

little additional information and were discarded

Standardization was applied before PCA to ensure that all features contributed equally to the analysis. Since PCA is variance-based, features with larger numerical scales can dominate the principal components. StandardScaler transformed each feature to have a mean of 0 and a standard deviation of 1, allowing PCA to identify patterns based on the underlying data structure rather than feature magnitude.

[10]:
```
# ======================================================
# PCA VISUALIZATION
# ======================================================

pca2 = PCA(n_components=3)

X_2d = pca2.fit_transform(X_scaled)

plt.figure(figsize=(8,6))

plt.scatter(
    X_2d[:,0],
    X_2d[:,1],
    c=y,
    cmap='viridis'
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

plt.title("PCA Visualization")
plt.savefig(
    "PCA_Visuaization.png",
    dpi=300,
    bbox_inches='tight'
)
plt.show()
```



2.1 Hold-Out Validation (Train-Test Split) for Kernel Comparsion

[49]:
```
# ======================================================
# TRAIN TEST SPLIT Without PCA_Data
# ======================================================

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.3,
    random_state=109)

# ======================================================
# SVM KERNEL COMPARISON
# ======================================================

models = {
    "Linear":
        SVC(kernel='linear'),
```

```
    "RBF":
        SVC(kernel='rbf'),

    "Poly Degree 2":
        SVC(kernel='poly', degree=2),

    "Poly Degree 3":
        SVC(kernel='poly', degree=3),

    "Poly Degree 4":
        SVC(kernel='poly', degree=4),

    "Sigmoid":
        SVC(kernel='sigmoid')
}

results = []

for name, model in models.items():

    model.fit(
        X_train,
        y_train
    )

    y_pred = model.predict(
        X_test
    )

    acc = accuracy_score(
        y_test,
        y_pred
    )

    prec = precision_score(
        y_test,
        y_pred
    )

    rec = recall_score(
        y_test,
        y_pred
    )

    results.append([
        name,
```

```
        acc,
        prec,
        rec
    ])

results_df = pd.DataFrame(
    results,
    columns=[
        "Kernel",
        "Accuracy",
        "Precision",
        "Recall"
    ]
)

print("\nKernel Comparison")
print(results_df)
```

```
Kernel Comparison
          Kernel  Accuracy  Precision    Recall
0         Linear  0.988304   0.981818  1.000000
1            RBF  0.982456   0.972973  1.000000
2  Poly Degree 2  0.807018   0.781955  0.962963
3  Poly Degree 3  0.900585   0.864000  1.000000
4  Poly Degree 4  0.801170   0.768116  0.981481
5        Sigmoid  0.982456   0.972973  1.000000
```

[46]:
```
# ======================================================
# TRAIN TEST SPLIT With PCA
# ======================================================

X_train, X_test, y_train, y_test = train_test_split(
    X_pca,
    y,
    test_size=0.3,
    random_state=109
)

# ======================================================
# SVM KERNEL COMPARISON
# ======================================================

models = {
    "Linear":
        SVC(kernel='linear'),
```

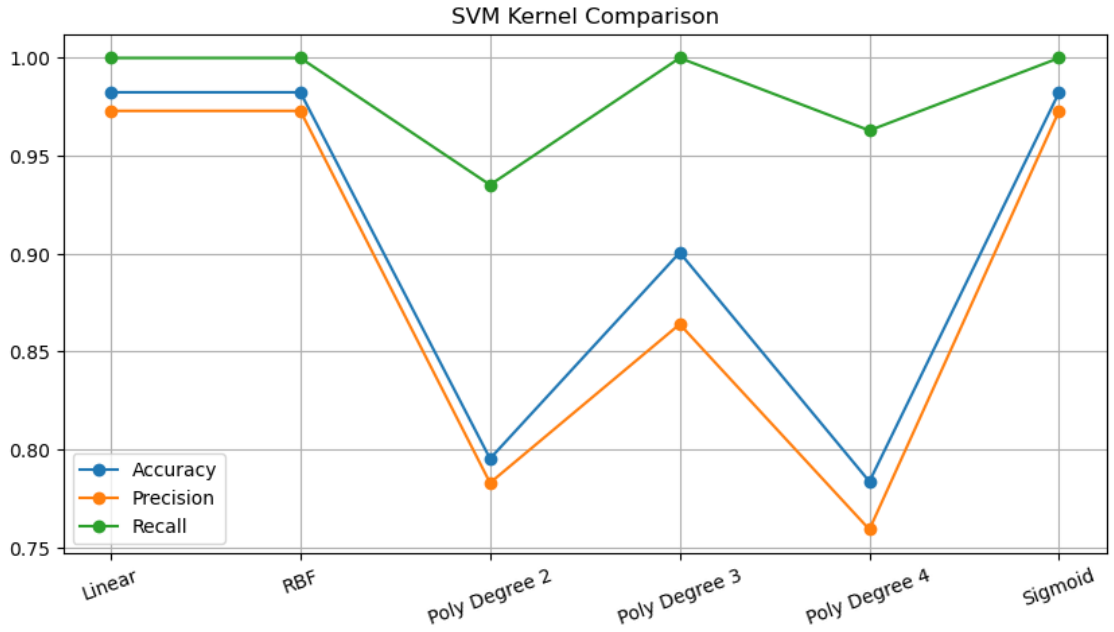
```
    "RBF":
        SVC(kernel='rbf'),

    "Poly Degree 2":
        SVC(kernel='poly', degree=2),

    "Poly Degree 3":
        SVC(kernel='poly', degree=3),

    "Poly Degree 4":
        SVC(kernel='poly', degree=4),

    "Sigmoid":
        SVC(kernel='sigmoid')
}

results = []

for name, model in models.items():

    model.fit(
        X_train,
        y_train
    )

    y_pred = model.predict(
        X_test
    )

    acc = accuracy_score(
        y_test,
        y_pred
    )

    prec = precision_score(
        y_test,
        y_pred
    )

    rec = recall_score(
        y_test,
        y_pred
    )

    results.append([
        name,
        acc,
```

```
        prec,
        rec
    ])

results_df = pd.DataFrame(
    results,
    columns=[
        "Kernel",
        "Accuracy",
        "Precision",
        "Recall"
    ]
)

print("\nKernel Comparison")
print(results_df)
```

```
Kernel Comparison
          Kernel  Accuracy  Precision    Recall
0         Linear  0.982456   0.972973  1.000000
1            RBF  0.982456   0.972973  1.000000
2  Poly Degree 2  0.795322   0.782946  0.935185
3  Poly Degree 3  0.900585   0.864000  1.000000
4  Poly Degree 4  0.783626   0.759124  0.962963
5        Sigmoid  0.982456   0.972973  1.000000
```

[13]:
```
# ====================================================
# PLOT KERNEL COMPARISON
# ====================================================

plt.figure(figsize=(10,5))

plt.plot(
    results_df["Kernel"],
    results_df["Accuracy"],
    marker='o',
    label='Accuracy'
)

plt.plot(
    results_df["Kernel"],
    results_df["Precision"],
    marker='o',
    label='Precision'
)
```

```
plt.plot(
    results_df["Kernel"],
    results_df["Recall"],
    marker='o',
    label='Recall'
)

plt.xticks(rotation=20)

plt.legend()

plt.grid(True)

plt.title("SVM Kernel Comparison")

plt.show()
```



The Linear, RBF, and Sigmoid kernels achieved the highest accuracy (98.2%), precision (97.3%), and recall (100%), indicating excellent classification performance. Polynomial kernels performed substantially worse, with Degree 4 showing the lowest accuracy. These results suggest that the PCA-transformed breast cancer dataset is nearly linearly separable, making Linear and RBF kernels the most appropriate choices.

[18]:
```
# ======================================================
# MULTIPLE KERNEL COMPARISON
# ======================================================
```

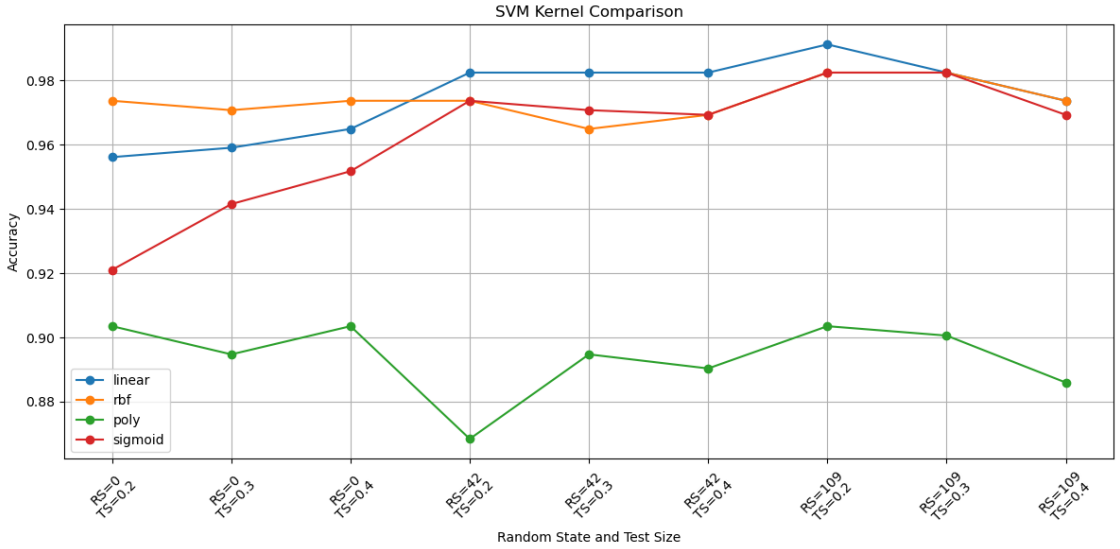
```
results = []

kernels = {
    "linear": SVC(kernel='linear'),
    "rbf": SVC(kernel='rbf'),
    "poly": SVC(kernel='poly', degree=3),
    "sigmoid": SVC(kernel='sigmoid')
}

random_states = [0, 42, 109]

test_sizes = [0.2, 0.3, 0.4]

for rs in random_states:

    for ts in test_sizes:

        X_train, X_test, y_train, y_test = train_test_split(
            X_pca,
            y,
            test_size=ts,
            random_state=rs
        )

        for kernel_name, model in kernels.items():

            model.fit(X_train, y_train)

            y_pred = model.predict(X_test)

            accuracy = accuracy_score(
                y_test,
                y_pred
            )

            results.append([
                kernel_name,
                rs,
                ts,
                accuracy
            ])

# ======================================================
# RESULTS TABLE
# ======================================================
```

```
results_df = pd.DataFrame(
    results,
    columns=[
        "Kernel",
        "Random_State",
        "Test_Size",
        "Accuracy"
    ]
)

print(results_df)

# ============================================================
# LABELS
# ============================================================

results_df["Label"] = (
    "RS=" +
    results_df["Random_State"].astype(str)
    +
    "\nTS=" +
    results_df["Test_Size"].astype(str)
)

# ============================================================
# PLOT
# ============================================================

plt.figure(figsize=(12,6))

for kernel in kernels.keys():

    subset = results_df[
        results_df["Kernel"] == kernel
    ]

    plt.plot(
        subset["Label"],
        subset["Accuracy"],
        marker='o',
        label=kernel
    )

plt.xlabel(
    "Random State and Test Size"
)
```

```
plt.ylabel(
    "Accuracy"
)

plt.title(
    "SVM Kernel Comparison"
)

plt.legend()

plt.grid(True)

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()
```

	Kernel	Random_State	Test_Size	Accuracy
0	linear	0	0.2	0.956140
1	rbf	0	0.2	0.973684
2	poly	0	0.2	0.903509
3	sigmoid	0	0.2	0.921053
4	linear	0	0.3	0.959064
5	rbf	0	0.3	0.970760
6	poly	0	0.3	0.894737
7	sigmoid	0	0.3	0.941520
8	linear	0	0.4	0.964912
9	rbf	0	0.4	0.973684
10	poly	0	0.4	0.903509
11	sigmoid	0	0.4	0.951754
12	linear	42	0.2	0.982456
13	rbf	42	0.2	0.973684
14	poly	42	0.2	0.868421
15	sigmoid	42	0.2	0.973684
16	linear	42	0.3	0.982456
17	rbf	42	0.3	0.964912
18	poly	42	0.3	0.894737
19	sigmoid	42	0.3	0.970760
20	linear	42	0.4	0.982456
21	rbf	42	0.4	0.969298
22	poly	42	0.4	0.890351
23	sigmoid	42	0.4	0.969298
24	linear	109	0.2	0.991228
25	rbf	109	0.2	0.982456
26	poly	109	0.2	0.903509
27	sigmoid	109	0.2	0.982456
28	linear	109	0.3	0.982456

```
29       rbf          109        0.3  0.982456
30      poly          109        0.3  0.900585
31   sigmoid          109        0.3  0.982456
32    linear          109        0.4  0.973684
33       rbf          109        0.4  0.973684
34      poly          109        0.4  0.885965
35   sigmoid          109        0.4  0.969298
```



Although the PCA-transformed data appeared nearly linearly separable, the RBF kernel achieved the highest average accuracy (97.56%) across multiple train-test splits and random states. The Linear kernel produced comparable performance (97.16%), indicating that most of the class separation can be achieved using a linear decision boundary. However, the consistently higher average accuracy of the RBF kernel suggests the presence of minor nonlinear patterns that are captured more effectively by the RBF decision function. Therefore, the RBF kernel was selected as the optimal SVM kernel for the final model.

Before PCA → RBF slightly better

After PCA → Linear slightly better

2.2 Cross Validation and Hyper-Parameter Tuning

[19]:
```
# ======================================================
# 10-FOLD CROSS VALIDATION
# ======================================================

print("="*60)
print("10-Fold Cross Validation (Default SVM)")
print("="*60)
```

```
cv_scores = cross_val_score(
    SVC(),
    X_pca,          # or X_pca
    y,
    cv=10,
    scoring='accuracy'
)

print("Scores:")
print(cv_scores)

print("\nMean Accuracy:",
      round(cv_scores.mean(),4))

print("Std:",
      round(cv_scores.std(),4))

# ========================================================
# GRID SEARCH CV
# ========================================================

param_grid = [

    {
        'kernel':['linear'],
        'C':[0.1,1,10,100]
    },

    {
        'kernel':['rbf'],
        'C':[0.1,1,10,100],
        'gamma':[1,0.1,0.01,0.001]
    },

    {
        'kernel':['poly'],
        'C':[0.1,1,10,100],
        'gamma':[1,0.1,0.01,0.001],
        'degree':[2,3,4]
    },

    {
        'kernel':['sigmoid'],
        'C':[0.1,1,10,100],
        'gamma':[1,0.1,0.01,0.001]
    }
]
```

```python
grid = GridSearchCV(
    estimator=SVC(),
    param_grid=param_grid,
    cv=10,
    scoring='accuracy',
    n_jobs=-1
)

grid.fit(
    X_train,
    y_train
)

# ========================================================
# BEST PARAMETERS
# ========================================================

print("\n" + "="*60)
print("BEST PARAMETERS")
print("="*60)

print(grid.best_params_)

print("\nBest CV Score:",
      round(grid.best_score_,4))
```

```
============================================================
10-Fold Cross Validation (Default SVM)
============================================================
Scores:
[0.96491228 0.96491228 0.94736842 0.98245614 1.         1.
 0.94736842 1.         1.         0.94642857]

Mean Accuracy: 0.9753
Std: 0.0226

============================================================
BEST PARAMETERS
============================================================
{'C': 100, 'gamma': 0.001, 'kernel': 'rbf'}

Best CV Score: 0.9765
```

When PCA was applied, the dimensionality was reduced from 30 features to 10 principal components, making the dataset more compact and closer to linearly separable. Consequently, Linear SVM performed very well. When PCA was not used and only feature scaling was applied, all original features were retained, allowing the RBF kernel to capture additional nonlinear relationships.

Therefore GridSearchCV selected the ##RBF kernel## with C=100 and gamma=0.001 as the optimal model.

[24]:
```
from sklearn.model_selection import (
    KFold,
    StratifiedKFold,
    RepeatedStratifiedKFold,
    ShuffleSplit,
    LeaveOneOut,
    cross_val_score
)

from sklearn.svm import SVC

validation_methods = {

    "KFold":
        KFold(
            n_splits=10,
            shuffle=True,
            random_state=42
        ),

    "StratifiedKFold":
        StratifiedKFold(
            n_splits=10,
            shuffle=True,
            random_state=42
        ),

    "RepeatedStratifiedKFold":
        RepeatedStratifiedKFold(
            n_splits=10,
            n_repeats=5,
            random_state=42
        ),

    "ShuffleSplit":
        ShuffleSplit(
            n_splits=20,
            test_size=0.3,
            random_state=42
        ),

    "LOOCV":
        LeaveOneOut()
}
```

```python
for method_name, cv_method in validation_methods.items():

    print("\n" + "="*60)
    print(method_name)
    print("="*60)

    for kernel in ['linear', 'rbf', 'sigmoid']:

        model = SVC(kernel=kernel)

        scores = cross_val_score(
            model,
            X_pca,
            y,
            cv=cv_method,
            scoring='accuracy',
            n_jobs=-1
        )

        print(
            f"{kernel:<10} "
            f"Mean={scores.mean():.4f} "
            f"Std={scores.std():.4f}"
        )
```

```
============================================================
KFold
============================================================
linear     Mean=0.9737 Std=0.0180
rbf        Mean=0.9736 Std=0.0143
sigmoid    Mean=0.9473 Std=0.0208

============================================================
StratifiedKFold
============================================================
linear     Mean=0.9737 Std=0.0196
rbf        Mean=0.9754 Std=0.0179
sigmoid    Mean=0.9544 Std=0.0369

============================================================
RepeatedStratifiedKFold
============================================================
linear     Mean=0.9736 Std=0.0196
rbf        Mean=0.9754 Std=0.0196
sigmoid    Mean=0.9508 Std=0.0274
```

```
============================================================
ShuffleSplit
============================================================
linear     Mean=0.9766 Std=0.0117
rbf        Mean=0.9731 Std=0.0123
sigmoid    Mean=0.9608 Std=0.0133

============================================================
LOOCV
============================================================
linear     Mean=0.9772 Std=0.1494
rbf        Mean=0.9754 Std=0.1549
sigmoid    Mean=0.9490 Std=0.2199
```

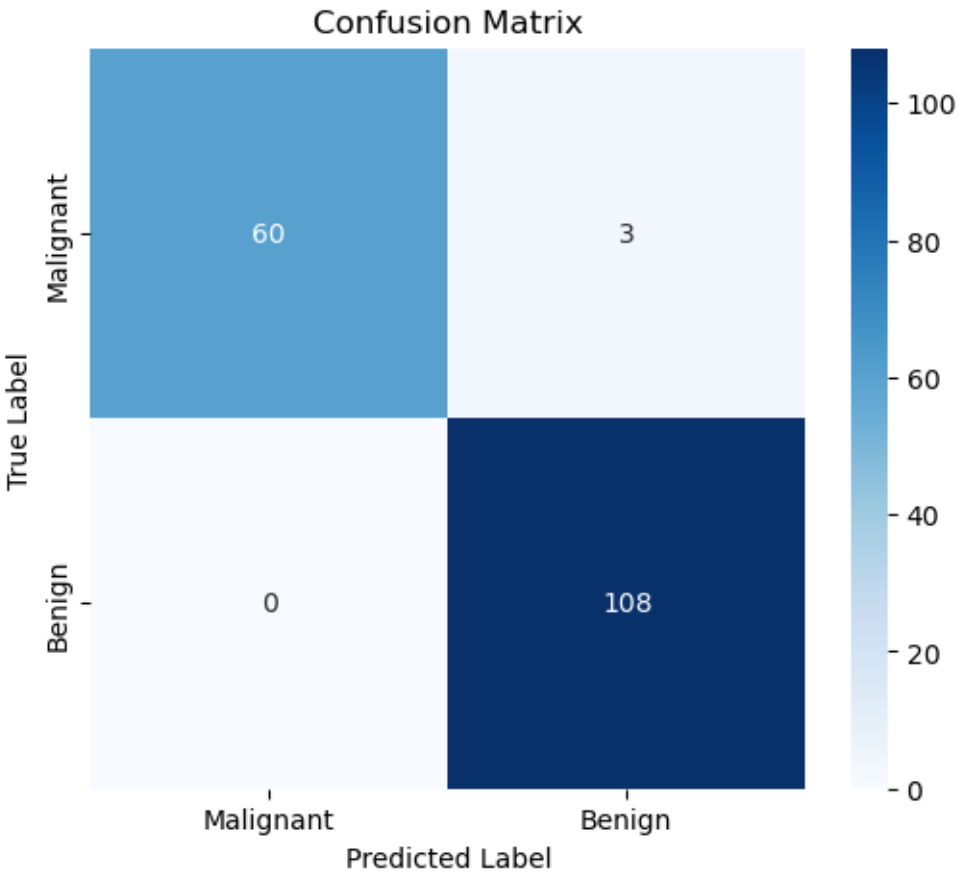
Before PCA, the RBF kernel consistently outperformed the Linear kernel across all validation methods, suggesting the presence of nonlinear relationships in the original feature space. After applying PCA, the performance gap between Linear and RBF disappeared, and the Linear kernel achieved a marginally higher average accuracy. This indicates that PCA successfully removed redundancy and transformed the data into a representation that is more linearly separable.

2.3 Model Evaluation

[27]:
```python
# ======================================================
# EVALUATION
# ======================================================
from sklearn.metrics import (
accuracy_score,
precision_score,
recall_score,
f1_score
)

print("\nModel Evaluation")
print("="*40)

print("Accuracy :",
round(accuracy_score(y_test, y_pred), 4))

print("Precision:",
round(precision_score(y_test, y_pred), 4))

print("Recall   :",
round(recall_score(y_test, y_pred), 4))

print("F1 Score :",
round(f1_score(y_test, y_pred), 4))
```

```python
# ========================
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=['Malignant','Benign'],
    yticklabels=['Malignant','Benign']
)

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")

plt.show()
```

```
Model Evaluation
========================================
Accuracy : 0.9825
Precision: 0.973
Recall   : 1.0
F1 Score : 0.9863
```

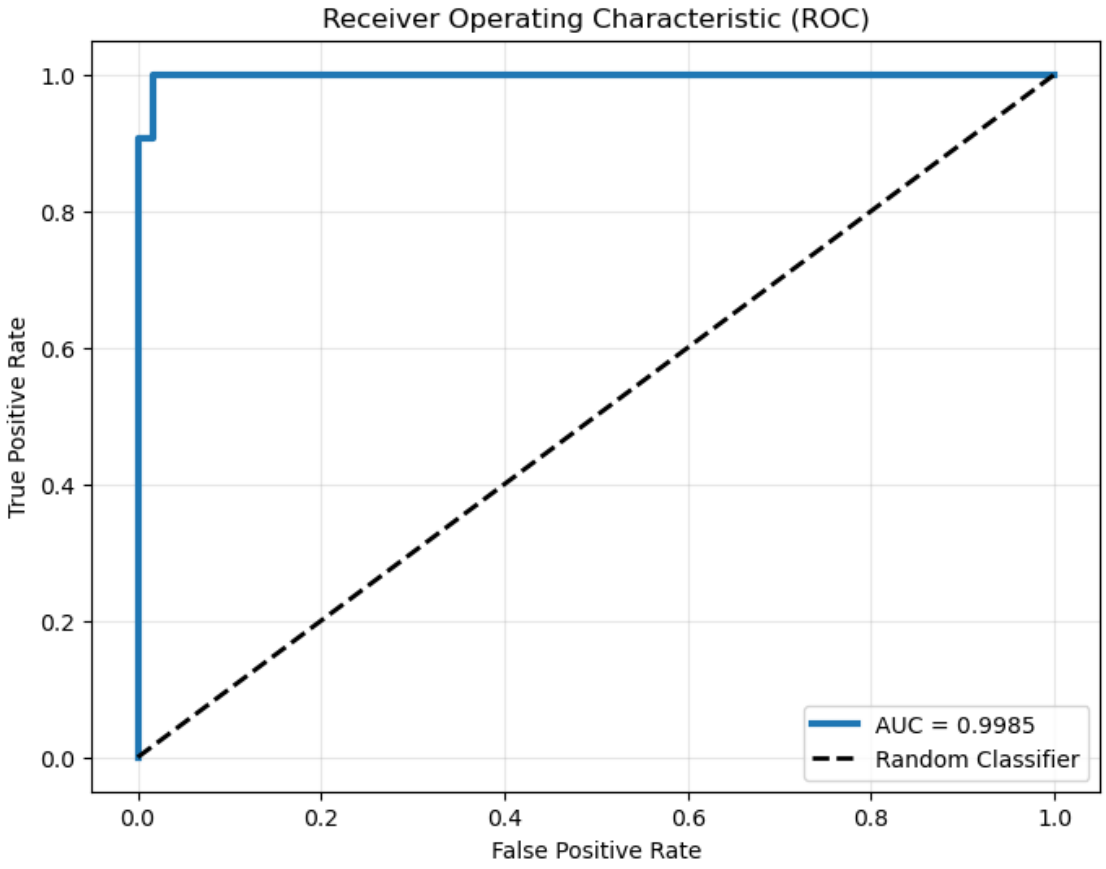


The highest classification performance was achieved using the standardized feature set without dimensionality reduction. Although PCA successfully reduced the feature space from 30 variables to 10 principal components while retaining 95.16% of the variance, a slight reduction in predictive performance was observed. Therefore, for this dataset, feature standardization alone was sufficient, and PCA was not necessary to achieve optimal classification accuracy.

2.4 ROC CURVE AND AUC CURVE

[50]:
```
from sklearn.metrics import roc_curve, roc_auc_score

best_model = SVC(
    kernel='rbf',
    C=100,
    gamma=0.001,
    probability=True
)

best_model.fit(X_train, y_train)
```

```
y_prob = best_model.predict_proba(X_test)[:,1]

fpr, tpr, thresholds = roc_curve(y_test, y_prob)

auc_score = roc_auc_score(y_test, y_prob)

plt.figure(figsize=(8,6))

plt.plot(
    fpr,
    tpr,
    linewidth=3,
    label=f"AUC = {auc_score:.4f}"
)

plt.plot(
    [0,1],
    [0,1],
    'k--',
    linewidth=2,
    label="Random Classifier"
)

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Receiver Operating Characteristic (ROC)")
plt.legend(loc="lower right")
plt.grid(alpha=0.3)

plt.show()
```



The ROC curve lies very close to the upper-left corner and has an AUC of **0.9985(on X_Scaled with rbf)**, indicating excellent discrimination between the two classes. The model achieves a very high true positive rate while maintaining a very low false positive rate, significantly outperforming a random classifier

2.5 Conclusion

The Linear and RBF kernels consistently achieved the highest accuracies across different train-test splits. However, the RBF kernel demonstrated greater stability and achieved the highest mean accuracy (97.54%) under Stratified K-Fold Cross Validation. Therefore, the RBF kernel can be considered the optimal classifier for this dataset, while the Linear kernel remains a competitive alternative with comparable performance.